A

Synopsis

on

Insurance Management System

in partial fulfillment of the requirement for the degree

of

Bachelor of Technology

In

COMPUTER SCIENCE AND ENGINEERING

Submitted by

Drishay Chauhan (2301330100084)
Himanshi Sharma (2301330100095)

Under the supervision of

Dr. Surya Prakash Sharma

Assistant Professor

NOIDA INSTITUTE OF ENGINEERING AND TECHNOLOGY

GREATER NOIDA

Index

Supervisor Sign:

INTRODUCTION

The rapid growth of digital technologies has transformed traditional business processes, including the insurance sector. However, many small-scale or traditional insurance systems still rely on manual processes for managing policies, customers, and claims. These methods are time-consuming, error-prone, and lack transparency, leading to inefficiencies in both user experience and administrative operations.

The proposed **Insurance Management System** is a web-based application developed using Java and the Spring Boot framework to automate and streamline insurance-related activities. The system provides a centralized platform where users can register, log in, view available insurance policies, purchase policies, and raise claims. At the same time, administrators can manage policies and handle claim approvals or rejections efficiently.

The application is designed using the Model-View-Controller (MVC) architecture, ensuring a clear separation of concerns and maintainable code structure. It uses Spring Data JPA for database interaction and JSP with Bootstrap for building a responsive and user-friendly interface.

This system focuses on improving usability, reducing manual effort, and ensuring data consistency. It incorporates key features such as role-based access control, session management, and validation mechanisms to prevent duplicate actions and unauthorized access.

Field of the Project:

- Web Development
- Enterprise Application Development
- Database Management Systems

Special Technical Terms:

- **MVC Architecture:** A design pattern that separates application logic into Model, View, and Controller.
- **Spring Boot:** A Java framework used to build web applications efficiently.
- **JPA (Java Persistence API):** A specification for managing relational data in Java applications.
- **Session Management:** Mechanism to maintain user authentication across requests.
- **Role-Based Access Control:** Restricting system access based on user roles (Admin/User).
- **CRUD Operations:** Create, Read, Update, Delete operations on data.

EXSISTING SYSTEM

Several insurance-related platforms and management systems exist today, ranging from traditional manual systems to modern digital platforms. These systems are designed to manage customer data, policies, and claim processing.

Traditional systems rely heavily on manual paperwork and human intervention. These methods are time-consuming, prone to errors, and lack transparency. Managing records manually also makes data retrieval and updates difficult.

Modern web-based insurance platforms provide features such as policy browsing, online registration, and claim tracking. Many insurance companies use enterprise-level systems that offer advanced functionalities, including customer management, analytics, and integration with external services. However, these systems are often complex, expensive, and not easily accessible for small-scale or academic use.

Additionally, many existing systems do not implement proper role-based access control or lack user-friendly interfaces. Some platforms focus primarily on administrative functionality, offering limited interaction for users. Others may require paid services or subscriptions to access advanced features.

Limitations of Existing Systems

- **Manual Processes:** Traditional systems depend on paperwork, leading to delays and errors.
- **Limited Accessibility:** Offline systems restrict users from accessing services anytime.
- **Complex Interfaces:** Many systems are difficult to use due to complicated design.
- **Lack of Role-Based Access:** Poor separation between user and admin functionalities.
- **Inefficient Claim Handling:** Claim processing is often slow and lacks transparency.
- **Data Inconsistency:** Lack of validation can lead to duplicate or incorrect data.
- **High Cost:** Advanced systems may require subscriptions or high infrastructure cost.

PROBLEM STATEMENT

The insurance sector still faces several challenges, especially in small-scale or traditional systems where processes are not fully digitized. Many existing systems rely on manual handling of data, leading to inefficiencies, delays, and increased chances of human error.

Users often find it difficult to manage insurance-related activities such as viewing policies, purchasing plans, and tracking claims due to the lack of a centralized and user-friendly platform. The absence of proper digital systems forces users to depend on physical documentation and offline communication, which reduces transparency and convenience.

Another major issue is the lack of role-based access control in basic systems. Without proper separation between user and administrator functionalities, there is a risk of unauthorized access and poor system management. Additionally, many systems do not implement proper validation mechanisms, which can lead to problems such as duplicate policy purchases and inconsistent data.

Claim processing is also a significant concern, as traditional methods are often slow and lack clear tracking, making it difficult for users to monitor the status of their claims. Furthermore, existing advanced systems are often complex and costly, making them unsuitable for small-scale implementations or academic purposes.

Key Problems Identified:

- Lack of a centralized digital platform for managing insurance operations
- Time-consuming manual processes and paperwork
- Difficulty in tracking policies and claims
- Absence of role-based access control
- Risk of data inconsistency and duplication
- Inefficient and non-transparent claim processing
- High complexity or cost of existing advanced systems

PROPOSED METHODOLOGY

The proposed Insurance Management System is a web-based application designed to automate and simplify insurance-related operations. The system is developed using Java and the Spring Boot framework, following the Model-View-Controller (MVC) architecture to ensure a structured and maintainable design.

The system is divided into multiple modules, each responsible for handling specific functionalities. The user interacts with the system through a web interface (JSP pages), and requests are processed by controllers, which communicate with the database using Spring Data JPA.

4.1 System Architecture and Modules

The system consists of the following modules:

- **User Module:** Allows users to register, log in, view available policies, purchase policies, and raise claims.
- **Admin Module:** Provides administrative functionalities such as adding new policies, deleting policies, and managing user claims.
- **Policy Management Module:** Handles the creation, display, and deletion of insurance policies.
- **Subscription Module:** Manages user-policy relationships when a user purchases a policy.
- **Claim Management Module:** Allows users to submit claims and enables administrators to approve or reject them.
- **Authentication Module:** Implements session-based login and role-based access control to ensure secure access.

4.2 System Flow

The working of the system follows a structured flow:

1. The user registers and logs into the system.
2. Based on the role (User/Admin), the system redirects to the respective dashboard.
3. Users can view available policies and purchase them.
4. Purchased policies are stored and displayed under “My Policies.”
5. Users can raise claims for their policies.
6. Admin reviews claims and approves or rejects them.
7. Admin can also add or delete policies.
8. All operations are stored and managed in the database.

4.3 Development Methodology

The system is developed using an **iterative and incremental approach**, where the project is built in stages:

- **Phase 1:** Implementation of basic user authentication (login and registration)
- **Phase 2:** Development of policy management and purchase functionality
- **Phase 3:** Integration of claim management system

- **Phase 4:** UI enhancement and validation logic implementation
- **Phase 5:** Testing, debugging, and final deployment

This approach allows gradual development, testing, and improvement of the system.

FEASIBILITY STUDY

The feasibility study evaluates the practicality and viability of developing the proposed Insurance Management System. The system has been analyzed from technical, operational, economic, and legal perspectives to ensure its successful implementation.

5.1 Technical Feasibility

The project is technically feasible as it uses widely available and well-supported technologies such as Java, Spring Boot, JSP, and MySQL. These technologies are stable, scalable, and suitable for developing web-based applications.

Spring Boot simplifies backend development by reducing configuration complexity, while Spring Data JPA enables efficient database interaction. The required development tools such as Eclipse/IntelliJ and Maven are easily accessible.

The system does not require any specialized hardware or software, making it easy to develop and deploy.

5.2 Operational Feasibility

The system is user-friendly and designed with a simple interface, making it easy for both users and administrators to operate. Minimal training is required to understand and use the system.

The role-based design ensures that users and administrators have clearly defined functionalities, improving usability and reducing confusion. The system automates major operations, reducing manual effort and improving efficiency.

5.3 Economic Feasibility

The project is economically feasible as it uses open-source technologies, eliminating licensing costs. Development can be done using standard computing resources without requiring expensive infrastructure.

Since the system is designed for academic and small-scale use, it does not involve significant financial investment, making it cost-effective.

5.4 Legal and Ethical Feasibility

The system does not handle sensitive financial transactions or confidential personal data beyond basic user information. All operations are performed within a controlled environment.

The project follows ethical practices by ensuring proper user authentication and data handling. It is intended for educational purposes and does not violate any legal regulations.

FACILITIES REQUIRED FOR PROPOSED WORK

6.1 Software Requirements

The development and execution of the Insurance Management System require the following software components:

- **Operating System:** Windows 10/11, macOS, or Linux
- **Programming Language:** Java
- **Framework:** Spring Boot
- **Architecture:** Model-View-Controller (MVC)
- **Database:** MySQL
- **ORM Tool:** Spring Data JPA / Hibernate
- **Frontend Technologies:** JSP, HTML, CSS, Bootstrap
- **IDE:** Eclipse / IntelliJ IDEA
- **Build Tool:** Maven
- **Web Server:** Apache Tomcat (embedded in Spring Boot)
- **Version Control:** Git and GitHub

6.2 Hardware Requirements

The system does not require high-end hardware and can be developed and run on standard computing systems:

- **Processor:** Intel i3 / AMD Ryzen 3 or higher
- **RAM:** Minimum 4 GB (8 GB recommended)
- **Storage:** At least 500 MB free space
- **System Type:** 64-bit Operating System
- **Internet Connection:** Required for downloading dependencies and database setup

CONCLUSION

The proposed Insurance Management System provides an efficient and structured solution for managing insurance-related operations through a web-based platform. The system successfully addresses the limitations of traditional and semi-digital methods by automating key processes such as user registration, policy management, policy purchase, and claim handling.

By implementing role-based access control, the system ensures that users and administrators have clearly defined functionalities, improving both security and usability. The use of the Spring Boot framework and MVC architecture enables a well-organized, scalable, and maintainable system design. Integration with a relational database using Spring Data JPA ensures reliable data storage and efficient data handling.

The system enhances user experience by providing a centralized platform where users can easily interact with insurance services, while administrators can efficiently manage policies and claims. Additionally, validation mechanisms such as preventing duplicate policy purchases and restricting deletion of policies in use improve data integrity and system reliability.

Overall, the project demonstrates the practical implementation of modern web development technologies and design principles. It serves as a strong foundation for building more advanced insurance systems and can be further enhanced with additional features such as secure authentication, payment integration, and advanced user interfaces in future developments.

REFERENCES

[1] Oracle Corporation, “Java Documentation,” [Online]. Available: https://docs.oracle.com/javase/. [Accessed: Apr. 2026].

[2] Pivotal Software, “Spring Boot Documentation,” [Online]. Available: https://spring.io/projects/spring-boot. [Accessed: Apr. 2026].

[3] Spring, “Spring Data JPA Reference Documentation,” [Online]. Available: https://docs.spring.io/spring-data/jpa/docs/current/reference/html/. [Accessed: Apr. 2026].

[4] Oracle Corporation, “MySQL Documentation,” [Online]. Available: https://dev.mysql.com/doc/. [Accessed: Apr. 2026].

[5] Bootstrap Team, “Bootstrap Documentation,” [Online]. Available: https://getbootstrap.com/docs/. [Accessed: Apr. 2026].

[6] TutorialsPoint, “Spring Boot Tutorial,” [Online]. Available: https://www.tutorialspoint.com/spring_boot/. [Accessed: Apr. 2026].

[7] Oracle Corporation, “Java EE (JSP & Servlet) Documentation,” [Online]. Available: https://docs.oracle.com/javaee/. [Accessed: Apr. 2026].

[8] Various Authors, “Web Development Tutorials and Projects,” GitHub and Medium, [Online]. Available: https://github.com/ and https://medium.com/. [Accessed: Apr. 2026].